

1. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    printf("Enter string: ");
    scanf("%s", s);

    int n = strlen(s);

    if (n > 0 && s[0] == 'a' && s[n - 1] == 'a')
        printf("Accepted");
    else
        printf("Rejected");

    return 0;
}
```

Output:

<pre>1 // Online C compiler to run C program online 2 #include <stdio.h> 3 #include <string.h> 4 5 int main() { 6 char s[100]; 7 printf("Enter string: "); 8 scanf("%s", s); 9 10 int n = strlen(s); 11 12 if (n > 0 && s[0] == 'a' && s[n - 1] == 'a') 13 printf("Accepted"); 14 else 15 printf("Rejected"); 16 17 return 0; 18 }</pre>	<pre>Enter string: abacda Accepted === Code Execution Successful ===</pre>
---	---

2. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with 0 and end with 1

Code:

```
#include <stdio.h>
#include <string.h>

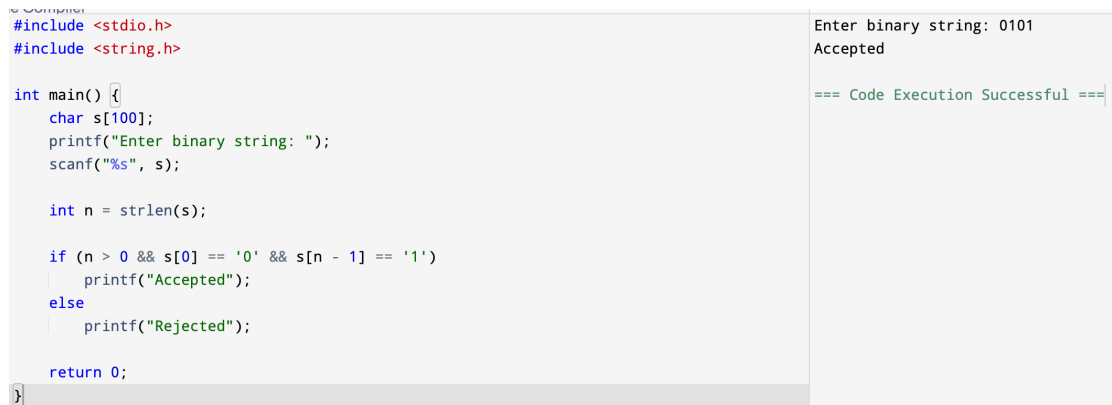
int main() {
    char s[100];
    printf("Enter binary string: ");
    scanf("%s", s);

    int n = strlen(s);

    if (n > 0 && s[0] == '0' && s[n - 1] == '1')
        printf("Accepted");
    else
        printf("Rejected");

    return 0;
}
```

Output:



The screenshot shows a C program being executed in a terminal. The code on the left defines a function `main` that prompts the user to enter a binary string. It checks if the string is non-empty, starts with '0', and ends with '1'. If all conditions are met, it prints "Accepted"; otherwise, it prints "Rejected". The output on the right shows the user input "0101", the program response "Accepted", and a confirmation message "=== Code Execution Successful ===".

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    printf("Enter binary string: ");
    scanf("%s", s);

    int n = strlen(s);

    if (n > 0 && s[0] == '0' && s[n - 1] == '1')
        printf("Accepted");
    else
        printf("Rejected");

    return 0;
}
```

Enter binary string: 0101
Accepted
=== Code Execution Successful ===

3. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0A1$ $A \rightarrow 0A \mid 1A \mid \epsilon$

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    printf("Enter string: ");
    scanf("%s", s);

    int n = strlen(s);

    if (n >= 2 && s[0] == '0' && s[n - 1] == '1')
        printf("String belongs to the CFG");
    else
        printf("String does not belong to the CFG");

    return 0;
}
```

Output:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    printf("Enter string: ");
    scanf("%s", s);

    int n = strlen(s);

    if (n >= 2 && s[0] == '0' && s[n - 1] == '1')
        printf("String belongs to the CFG");
    else
        printf("String does not belong to the CFG");

    return 0;
}
```

```
Enter string: 001
String belongs to the CFG

=== Code Execution Successful ===
```

4. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$$

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, n;

    printf("Enter string: ");
    scanf("%s", s);

    n = strlen(s);

    for(i = 0; i < n / 2; i++) {
        if(s[i] != s[n - 1 - i]) {
            printf("String does not belong to the CFG");
            return 0;
        }
    }

    printf("String belongs to the CFG");
    return 0;
}
```

Output:

<pre>#include <stdio.h> #include <string.h> int main() { char s[100]; int i, n; printf("Enter string: "); scanf("%s", s); n = strlen(s); for(i = 0; i < n / 2; i++) { if(s[i] != s[n - 1 - i]) { printf("String does not belong to the CFG"); return 0; } } printf("String belongs to the CFG"); return 0; }</pre>	<pre>Enter string: 101 String belongs to the CFG === Code Execution Successful ===</pre>
--	---

5. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$$S \rightarrow 0S0 \mid A \quad A \rightarrow 1A \mid \epsilon$$

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i = 0, j, c = 0;
    printf("Enter string: ");
    scanf("%s", s);
    j = strlen(s) - 1;
    while (i < j && s[i] == '0' && s[j] == '0') {
        c++;
        i++;
        j--;
    }
    while (i <= j) {
        if (s[i] != '1') {
            printf("String does not belong to CFG");
            return 0;
        }
        i++;
    }
    printf("String belongs to CFG");
    return 0;
}
```

Output:

<pre>ie Compiler #include <stdio.h> #include <string.h> int main() { char s[100]; int i = 0, j, c = 0; printf("Enter string: "); scanf("%s", s); j = strlen(s) - 1; while (i < j && s[i] == '0' && s[j] == '0') { c++; i++; j--; } while (i <= j) { if (s[i] != '1') { printf("String does not belong to CFG"); return 0; } i++; } printf("String belongs to CFG"); return 0; }</pre>	<pre>Enter string: 0110 String belongs to CFG === Code Execution Successful ===</pre>
---	--

6. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S1 \mid \epsilon$

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i = 0, c0 = 0, c1 = 0;

    printf("Enter string: ");
    scanf("%s", s);

    while (s[i] == '0') {
        c0++;
        i++;
    }

    while (s[i] == '1') {
        c1++;
        i++;
    }

    if (s[i] == '\0' && c0 == c1)
        printf("String belongs to CFG");
    else
        printf("String does not belong to CFG");
    return 0;
}
```

Output:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i = 0, c0 = 0, c1 = 0;
    printf("Enter string: ");
    scanf("%s", s);
    while (s[i] == '0') {
        c0++;
        i++;
    }
    while (s[i] == '1') {
        c1++;
        i++;
    }
    if (s[i] == '\0' && c0 == c1)
        printf("String belongs to CFG");
    else
        printf("String does not belong to CFG");
    return 0;
}
```

Enter string: 0011
String belongs to CFG

=== Code Execution Successful ===

7. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow A101A, A \rightarrow 0A \mid 1A \mid \epsilon$

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];

    printf("Enter string: ");
    scanf("%s", s);

    if (strstr(s, "101"))
        printf("String belongs to CFG");
    else
        printf("String does not belong to CFG");

    return 0;
}
```

Output:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];

    printf("Enter string: ");
    scanf("%s", s);

    if (strstr(s, "101"))
        printf("String belongs to CFG");
    else
        printf("String does not belong to CFG");

    return 0;
}
```

```
Enter string: 101
String belongs to CFG

=== Code Execution Successful ===
```

8. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with b and end with a

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    printf("Enter string: ");
    scanf("%s", s);

    int n = strlen(s);

    if (n > 0 && s[0] == 'b' && s[n-1] == 'a')
        printf("String Accepted");
    else
        printf("String Rejected");

    return 0;
}
```

Output:

<pre>#include <stdio.h> #include <string.h> int main() { char s[100]; printf("Enter string: "); scanf("%s", s); int n = strlen(s); if (n > 0 && s[0] == 'b' && s[n-1] == 'a') printf("String Accepted"); else printf("String Rejected"); return 0; }</pre>	<pre>Enter string: bacda String Accepted === Code Execution Successful ===</pre>
---	---

9. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with 0 and end with 1

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    printf("Enter binary string: ");
    scanf("%s", s);

    int n = strlen(s);

    if (n > 0 && s[0] == '0' && s[n-1] == '1')
        printf("String Accepted");
    else
        printf("String Rejected");

    return 0;
}
```

Output:

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    printf("Enter binary string: ");
    scanf("%s", s);

    int n = strlen(s);

    if (n > 0 && s[0] == '0' && s[n-1] == '1')
        printf("String Accepted");
    else
        printf("String Rejected");

    return 0;
}
```

```
Enter binary string: 0101
String Accepted

=== Code Execution Successful ===
```

10. Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ϵ -moves.

Code:

```
#include <stdio.h>
int e[10][10], n, v[10];
void closure(int s) {
    int i;
    v[s] = 1;
    printf("q%d ", s);
    for(i = 0; i < n; i++)
        if(e[s][i] && !v[i])
            closure(i);
}
int main() {
    int i, j;
    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter epsilon transition matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &e[i][j]);
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++)
            v[j] = 0;
        printf("\nE-Closure(q%d) = { ", i);
        closure(i);
        printf("}");
    }
    return 0;
}
```

Output:

<pre>#include <stdio.h> int e[10][10], n, v[10]; void closure(int s) { int i; v[s] = 1; printf("q%d ", s); for(i = 0; i < n; i++) if(e[s][i] && !v[i]) closure(i); } int main() { int i, j; printf("Enter number of states: "); scanf("%d", &n); printf("Enter epsilon transition matrix:\n"); for(i = 0; i < n; i++) for(j = 0; j < n; j++) scanf("%d", &e[i][j]); for(i = 0; i < n; i++) { for(j = 0; j < n; j++) v[j] = 0; printf("\nE-Closure(q%d) = { ", i); closure(i); printf("}"); } return 0; }</pre>	<pre>Enter number of states: 3 Enter epsilon transition matrix: 0 1 0 0 0 1 0 0 0 E-Closure(q0) = { q0 q1 q2 } E-Closure(q1) = { q1 q2 } E-Closure(q2) = { q2 } === Code Execution Successful ===</pre>
---	---

11. Write a C program to find ϵ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ϵ -moves.

Code:

```
#include <stdio.h>
int a[10][10], v[10], n;
void eclosure(int s)
{
    int i;
    v[s] = 1;
    printf("q%d ", s);
    for(i = 0; i < n; i++)
        if(a[s][i] == 1 && !v[i])
            eclosure(i);
}
int main()
{
    int i, j;
    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter epsilon transition matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
            v[j] = 0;
        printf("\nE-Closure(q%d) = { ", i);
        eclosure(i);
        printf("}");
    }
    return 0;
}
```

Output:

```
#include <stdio.h>
int a[10][10], v[10], n;
void eclosure(int s)
{
    int i;
    v[s] = 1;
    printf("q%d ", s);
    for(i = 0; i < n; i++)
        if(a[s][i] == 1 && !v[i])
            eclosure(i);
}
int main()
{
    int i, j;
    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter epsilon transition matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
            v[j] = 0;
        printf("\nE-Closure(q%d) = { ", i);
        eclosure(i);
        printf("}");
    }
    return 0;
}
```

Enter number of states: 3
Enter epsilon transition matrix:
0 1 0
0 0 1
0 0 0

E-Closure(q0) = { q0 q1 q2 }
E-Closure(q1) = { q1 q2 }
E-Closure(q2) = { q2 }

=== Code Execution Successful ===